

F1R3COMB-MAT

The rho combinators as sparse linear algebra:
a Rust implementation of the matrix representation,
the bulk-synchronous machine, and the transfer operator
Specification v0.5 — companion to F1R3COMB v0.6

F1R3FLY.io

23 September 2026

Abstract

F1R3COMB-MAT is the third stage of the route from the rho calculus to a data-parallel device: it takes a F1R3COMB term to the matrix representation of [1] and runs it on a bulk-synchronous machine whose step is a maximal independent set of redexes. Phase two of the translation erects each run-time instance at an address, and within the discipline draft 3’s correctness proof needs, that takes a single-premise rule of multiplicity at least two [4, Prop. 6.17, Lem. 6.19]. The implementation path uses the context-labelled combinator $\text{inst}(a, f, C) \mid m(a, v) \longrightarrow m(f, @C[v])$.

F1R3COMB v0.6 now agrees [3, Dec. 5.19]: `inst` is its default, and the curried path-indexed constructors are kept behind `curried-erection` as the build [5] can read. This version brings F1R3COMB-MAT into line with it. `inst` is the machine’s erection with no feature to enable. Curried artefacts run on the host behind the same `curried-erection` feature, for the cross-scheme obligations F1R3COMB has added [3, Obl. 11.5, Obl. 11.6]. The literal erection remains a negative control. The context node, `inst` and the hole take F1R3COMB’s tags, with every hole collapsed to one reserved name on import, which is sound because `inst` has a single input.

Two points of F1R3COMB v0.6’s rationale for `inst` read differently from the machine side, and are returned as requests. The depth argument is right as a count of bulk-synchronous steps — 2 per instance under `inst` against $\lceil \log_2 n \rceil + 2$ under `curried`, and 7 against 10 per unfolding of the recursion combinator — but not as the band of an upper-triangular operator, since every compiled program with an input contains `e` and has no such operator. And the concern that `inst` makes the rule matrix program-dependent [3, §15, item 5] does not arise here: this representation holds the context as a name in the state, so `inst` is one rule for every program. The curried scheme’s limited reach (E7) now matters more than it did, because the curried build is the one the logic reads and the one F1R3COMB’s new obligations run on the whole corpus. Key words MUST, MUST NOT, SHOULD, MAY are used in the sense of RFC 2119.

Contents

1	Purpose and scope	2
1.1	The route, and where this document sits on it	2
1.2	What exists today	3
1.3	Nomenclature	4
1.4	Goals	4
2	Position relative to F1R3COMB	5
2.1	Decisions	5

3	Phase two, with the evidence	6
3.1	The obstruction, now in the paper	6
3.2	Three arms	6
3.3	Templates	7
3.4	Marking, and when a crossing is sanctioned	8
3.5	Provenance	8
4	The representation	8
5	The rule table	9
6	Redex finding, selection, firing	9
7	Addresses and freshness	9
8	The operator layer	10
9	Instrumentation	10
10	Architecture	10
10.1	The device seam	10
11	Correctness obligations and testing	11
12	Errors and diagnostics	11
13	Performance, portability, determinism	11
14	What the implementation asks of the papers	12
15	Open questions	13
16	Delivery	14
17	Changes from v0.4	14

1 Purpose and scope

1.1 The route, and where this document sits on it

Stage	What it does	Specified in
1	K0 text to a canonical normal form	[6]
2	Normal form to combinator term, in two phases	F1R3COMB v0.6 [3]
3	Combinator term to matrix representation; the bulk-synchronous machine; the transfer operator	this document
4	The machine’s kernels on a device	Section 10.1, as a contract

This version aligns the machine with F1R3COMB v0.6, which takes `inst` as its default. Otherwise it leaves v0.3’s changes to phase two, and the rest of v0.2 [2], as they were. Where a requirement below says “as v0.2”, v0.2’s text is normative; where the two differ, this document governs. Two properties govern everything below. The machine computes exactly one step per state, seed and configuration, so a host and a device can be compared trace for trace ([1, P4]); and it runs what stage two emits, including every instance of every unit, at the width the program has ([1, Princ. 10.4]).

1.2 What exists today

Artefact	What it gives this project
<code>rho-combinators-draft3</code>	The calculus and its presentations (§3.2–3.3); the constructor family (Def. 3.8, Rem. 3.9); erection at a computed name (Rem. 3.11); phase one (§6.2) and phase two (§6.3, Def. 6.8, Ex. 6.9); the discipline and the permutation lemma (Def. 6.10, Lem. 6.11); freshness by size (Lem. 6.14); <i>new</i> : the literal erection’s failure (Ex. 6.16), multiplicity (Prop. 6.17, Def. 6.18, Lem. 6.19), the marking (Rem. 6.20), the deciding term (Ex. 6.21), the two schemes and the recommendation (§6.7, Rem. 6.22); the weighted count (Lem. 9.2)
<code>f1r3comb-spec-v06</code>	The term type and shape table; tags, now with inst , the context node and the hole (Req. 4.11); phase two as a compiler; the literal erection forbidden (Req. 5.15); the marking analysis (Req. 5.17); <i>new</i> : inst by default and curried behind curried-erection (Dec. 5.19, Rem. 5.22, Rem. 5.23); the context node (Req. 5.24); filling a context (Req. 5.25); the curried build (Req. 5.27); both schemes on the corpus, and contexts round-trip (Obl. 11.5, 11.6)
<code>rhocomb-matrix v1.2</code>	The state space, incidence, step and operator; §14, phase two seen from the machine
<code>rhocombmat.py v1.3</code> , <code>phase2.py</code> , <code>phase2-results.txt</code>	The reference machine with the generated table, inst , and the curried constructor cons* ; the three-arm experiments E5–E7 of Section 3. The repository still holds the two-arm versions (v1.2); the three-arm files accompany this specification and are asked to be committed (Section 14, item 1)
<code>rhocomb-logic rev. 2</code>	Components (Def. 2.9); drop elimination (Lem. 2.8); injective quotation (Prop. 2.10); the weighted count as a formula (Def. 4.2, Prop. 6.17)

1.3 Nomenclature

Term	Meaning in this document
<i>term, shape table, family</i>	As in v0.2: a closed drop-free Term ; F1R3COMB's table of shapes; the constructors a program uses, $\mathcal{F}(P)$
<i>row width, w</i>	The largest arity in the program's signature
<i>name id, row, row ref</i>	As in v0.2
<i>address, leaf, static channel, unit, instance</i>	As in v0.2 and [3, §1.3]
<i>scoped, marked</i>	A bound name allocated beneath the address; a name carrying an instance-specific value, in the sense of [4, Rem. 6.20]
<i>template</i>	An interned name over the reserved HOLE : the thing an erecting rule instantiates at an address
<i>scheme</i>	How a unit is erected: <i>curried</i> [4, §6.7], <i>inst</i> [4, §6.7], or <i>literal</i> [4, Def. 6.8]
<i>provenance</i>	The instance a row belongs to, as far as the machine can tell; metadata, never semantics
<i>mixed</i>	An atom mentioning scoped names of two addresses

1.4 Goals

1. A total, lossless, round-trippable map from F1R3COMB terms to the matrix representation, for every shape F1R3COMB's shape table generates and every scheme F1R3COMB emits.
2. Redex finding as joins derived from the generated rule table, with three finders that agree.
3. A bulk-synchronous step defined bit-exactly.
4. Phase-two images erected by the context-labelled combinator **inst** run correctly, with F1R3COMB's allocator, freshness checks and marking; curried artefacts run on the host for F1R3COMB's cross-scheme obligations; the literal scheme is available as a negative control.
5. The instruments that bear on F1R3COMB's open choices: width with erection separated, per-scheme cost, cross-instance mixing, and a brute-force oracle for the marking analysis.
6. On the destructor-free sublattice, the transfer operator, scoped honestly.
7. F1R3COMB's portability envelope.

Non-goals are those of v0.2: device code (a contract only), presentation B, weights, the logic, the history unfolding, intern-table collection, and phase one and two themselves. F1R3COMB-MAT takes a **Term** and MUST NOT depend on **f1r3comb-lower** or **f1r3comb-ir**.

2 Position relative to F1R3COMB

	F1R3COMB machine (<code>f1r3comb-run</code>)	F1R3COMB-MAT (<code>f1r3comb-par</code>)
State	CAMPF1R3 store keyed by <code>Hash32</code>	shape tables of <code>NameId</code> rows, width w
Name identity	content hash of the encoding	structural intern in-run, content hash at the boundary
Event	one comm	one step, a maximal independent set
Resolver	store order key; adversarial profile [3, Obl. 11.2]	priority function; cross-instance profile
Rules	generated from the shape table	generated from the same table
Erection	<code>inst</code> by default; curried under <code>curried-erection</code>	<code>inst</code> by default; curried on the host under <code>curried-erection</code> ; literal as a negative control
Addresses	allocator [3, Req. 9.6]	the same allocator

2.1 Decisions

Decision 2.1 (Presentation A only). Unchanged from v0.2. Under B rows are ragged and (G1) of [1, §2.1] fails; the A/B comparison [3, Obl. 11.12] is run on `f1r3comb-run`.

Decision 2.2 (A store row holds the quotation of its payload). Unchanged from v0.2: the row for $q(a, p)$ is $(q, \text{id}(a), \text{id}(@p))$, lossless under A, with the shape tag carrying the m/q distinction, and `consq` uniform with every other member.

Decision 2.3 (Decode by arena read). Unchanged from v0.2, and still at odds with [3, Req. 9.4] (Section 14, item 10).

Decision 2.4 (The rule table is generated from F1R3COMB’s shape table). Unchanged from v0.2. The generator lives in `f1r3comb-term`; F1R3COMB-MAT lists no rules of its own.

Decision 2.5 (`inst` is the erection; curried behind the same feature as F1R3COMB). The implementation path erects by the context-labelled combinator `inst`, and so does F1R3COMB v0.6 by default [3, Dec. 5.19]. F1R3COMB-MAT MUST support `inst` unconditionally. Every measurement that bears on the path — width, cost per instance, conformance, the device conformance corpus — MUST be made on `inst` artefacts, and the device contract targets `inst` alone (Requirement 10.1). F1R3COMB-MAT runs whatever scheme the artefact header names, and under `curried-erection`, the feature name F1R3COMB uses, MUST also run curried artefacts on the host. That support exists for F1R3COMB’s cross-scheme obligations [3, Obl. 11.5, Obl. 11.6] and for the logic tooling’s build [3, Req. 5.27]; it is not a second execution path. It MUST support the literal scheme under a test-only feature `literal-erection`, which no product crate enables, because [3, Obl. 11.3] requires the broken construction as a negative control and F1R3COMB is forbidden to emit it [3, Req. 5.15]. It MUST accept units that F1R3COMB’s marking analysis has compiled with the fixed-arity family [3, Req. 5.17], reading the marking from the header (Section 3.4).

Decision 2.6 (One kernel, whose primary client is `inst`; contexts are data). A curried constructor $\text{cons}_A^*[w](t, f)$ and an $\text{inst}(a, f, C)$ both consume one message and deliver the quotation of a template instantiated at its payload. F1R3COMB-MAT MUST implement both by one rule shape with one encode, `Instantiate{template, with}`, and distinguish them by a validity class on the template (Requirement 3.2). F1R3COMB v0.6 records that `inst` adds one term operation to its CAMPF1R3 machine [3, Rem. 9.2]; on this machine there is nothing to add, since the curried scheme’s leaves are built by the same substitution and both schemes are the same kernel.

Because the template is the third column of the row, a context is a name in the state, not part of the rule. `inst` is therefore one rule for every program, and the rule table is independent

of the program's units; so is `cons*`'s. This answers the second half of [3, §15, item 5] for this representation (Section 14, item 4).

3 Phase two, with the evidence

3.1 The obstruction, now in the paper

[4, §6.6] shows the literal erection fails the discipline on a single instance (Ex. 6.16), that the obvious repair regresses, and that the obstruction is structural: under the discipline extended to all rules every reachable atom holds at most one occurrence of an address (Prop. 6.17), so erecting any atom with two scoped names needs a rule of multiplicity at least two (Lem. 6.19). v0.2's Section 3.2 is discharged by this and F1R3COMB v0.5 withdraws its two-stage construction [3, Rem. 5.16]. [4, §6.7] supplies two single-premise rules of the required multiplicity:

$$\text{cons}_A^*[w](t, f) \mid m(t, \sigma) \longrightarrow m(f, @A(\sigma w_1, \dots, \sigma w_n)), \quad \text{inst}(a, f, C) \mid m(a, v) \longrightarrow m(f, @C[v]).$$

3.2 Three arms

[3, Rem. 5.20] still records that the committed measurements compare the literal scheme with `inst` and have no curried arm, and that a third arm would let the figures be compared directly rather than argued. The three-arm `phase2.py` that accompanies this specification runs it, with the curried scheme implemented as [3, Rem. 5.21] describes it: no allocation gadget, one `d` per erected atom fanning the address out on static channels, one `cons*` and one `e` per atom. Its templates allow a static name at an argument position, as [4, Ex. 6.16]'s `d(a, p0, q1)` requires.

E5. Unit $\{\text{fw}(z_1, z_2), \text{k}(z_1)\}$; runs with a mixed atom, of 50				
instances	2	4	8	16
literal, maximal progress / sequential	29 / 16	48 / 38	50 / 49	50 / 50
curried, either schedule	0	0	0	0
<code>inst</code> , either schedule	0	0	0	0
parallel steps to erect: literal 5, curried 3, <code>inst</code> 2				

E6. D_x of [4, Ex. 6.9], 200 unfoldings, per unfolding			
scheme	parallel steps	sequential steps	new names
curried	10	32.0 (29–35)	22
<code>inst</code>	7	10.0 (9–11)	16

E7. Store $q(b, @(fw(c_1, c_3) \mid k(c_2)))$, b static, c_i scoped; mixed stores		
scheme	2 instances	8 instances
curried, payload joined by <code>cons</code> _l	44 / 100	347 / 400
<code>inst</code>	0 / 100	0 / 400

Three conclusions, each a requirement below.

Both conforming schemes are correct for top-level atoms. E5 answers [3, Rem. 5.20]'s question on conformance: no mixing under either, at any count, under either schedule. Conformance does not separate them; depth, cost and reach do.

`inst` erects in fewer bulk-synchronous steps. Two per instance, against $\lceil \log_2 n \rceil + 2$ for a curried unit of n atoms (the duplicator tree, then `cons*`, then `e`); 7 against 10 per unfolding of D_x . This is [3, Rem. 5.22]'s depth argument, measured in the unit it holds in (Section 14, item 2).

The three together are why the implementation path uses `inst`.

On this machine `inst` also does less work. [3, Rem. 5.22] holds that the work figure does not change between the schemes: `inst` spends $O(|\text{unit}|)$ in one step, recomputing encodings up the rebuilt term, and `curried` spends the same across its steps. Structural interning computes no hashes (Requirement 4.1), and the substitution visits only names that mention the hole (Requirement 3.5), so the work here is names interned: 16 per unfolding under `inst` against 22 under `curried`, where each `cons*` builds its leaves by d interns from the address. Both schemes are charged by names built, as [3, Req. 5.25] requires, and on that measure `inst` is cheaper here.

The `curried` scheme does not reach every store. Its template places the address at argument positions of one atom. A gate store whose body mentions scoped names in two atoms — which is the body of any input whose bound name occurs in two atoms of its continuation — needs its payload assembled from two address-bearing pieces, and the only rule that assembles them is a join. The store atom then mentions two marked names, which is [4, Rem. 6.20]’s own criterion, so Lemma 6.19 applies to it; the multiplicity it needs is beneath a quotation, which the `curried` rule does not supply. [3, Req. 4.9]’s recipe (build the payload by `conse`, feed it to `consq`) conforms exactly when the store’s subject is static and the payload has one scoped atom, which is the case of [4, Ex. 6.9]; E6 uses it there. E7 is the case beyond it.

Hazard 3.1 (Curried stores). Under `curried-erection`, a unit whose gate store’s payload mentions scoped names in more than one atom, or beneath a further quotation, cannot be erected within the discipline, and a compiler that falls back to a join produces the mixing E7 measures. This matters more under F1R3COMB v0.6 than before: the `curried` build is the one the logic reads [3, Req. 5.27], and [3, Obl. 11.5, Obl. 11.6] run it on the whole corpus and compare it with `inst`, which such units will fail for reasons of the scheme, not the compiler. Whether that mixing is observable is the deciding-term question of [4, Ex. 6.21] applied to stores, which the marking analysis does not yet ask (Section 14, item 7). F1R3COMB-MAT MUST detect such stores at deploy under the `curried` scheme and report them as `comb-mat-curried-store`, a warning, naming the unit.

3.3 Templates

Requirement 3.2 (One representation for both schemes). A template MUST be an interned name over the reserved `NameId::HOLE`. Both erecting rules MUST be rows of arity three — (t, f, T) for `cons*`, (a, f, C) for `inst` — with the template in the third column, so the row width is four under either scheme. A `cons*` template MUST satisfy the `curried` validity class: it is a single atom, and each argument is either a path beneath `HOLE` (a L/R spine) or a name that does not mention `HOLE`. The check runs at import and at every firing in debug builds; a violation is `comb-mat-template`. An `inst` template is any name over `HOLE`.

Requirement 3.3 (Contexts and holes on import and export). F1R3COMB’s context node (tag `0x61`) and hole (tag `0x62`, carrying a canonical index) [3, Req. 4.11, Req. 5.24] MUST be imported as a template over the single reserved `HOLE`, every hole index collapsed. This is sound because `inst` has one input, so every hole is filled with the same name, and it is necessary for structural interning: two occurrences of the same leaf beneath distinct hole indices would otherwise be distinct names, and a template would share nothing. Export MUST renumber holes in encoding order, so that export–import and import–export round-trip against F1R3COMB’s encoding (Section 14, item 6).

Requirement 3.4 (The hole). As v0.2: `HOLE` has no counterpart in the term type outside a template position, is rejected by import anywhere else (`comb-mat-hole`), and no rule’s conclusion may contain it. Its encoding is F1R3COMB’s hole tag `0x62` (Requirement 3.3); `inst` is `0x60`. `cons*` has no tag yet (Section 14, item 7).

Requirement 3.5 (Instantiation). The encode **MUST** compute $T\{v/\text{HOLE}\}$ over the interned form, hereditarily beneath quotation, with a per-firing memo keyed by name id. The set of names in T that mention **HOLE** **MUST** be precomputed once per template, at import, and the traversal **MUST** visit only those, reusing every other name of T unchanged; this is [3, Req. 5.25]’s precomputed hole positions in interned form, and its cost is the number of hole-bearing names of T , not the size of the unit. Result names are interned in canonical order (Requirement 6.1). The report **MUST** attribute the names built to the scheme, which is the work figure of E6.

Requirement 3.6 (The measured costs are regressions, **inst** first). **f1r3comb-check** **MUST** reproduce E6 as laws over the number of unfoldings to 10^4 : 7 parallel steps and 16 names per unfolding under **inst**, with a sequential mean of 10, as the primary regression, matching [3, Obl. 11.3]; and 10, 22 and 32 under **curried-erection**. A golden file **MUST NOT** be compared across schemes [3, Dec. 5.19].

3.4 Marking, and when a crossing is sanctioned

[3, Req. 5.17] compiles a unit with the fixed-arity family when no atom of its phase-one image mentions two marked names, since crossings there permute which instance handles which value and every answer is right. On such a unit this machine will observe crossings — that is what the analysis licenses — and must not report them as defects.

Requirement 3.7 (The header’s marking is read, and checked). **F1R3COMB-MAT** **MUST** read the marking from the artefact header and carry, for each scoped name, whether it is marked. A row mentioning scoped names of two addresses is *mixed*; it is *sanctioned* if at most one of those names is marked and *unsanctioned* otherwise. Unsanctioned mixed rows **MUST** raise a trace event at every trace level and be counted in the run report; under a conforming scheme with a correct marking there are none, and the test suite **MUST** assert it under **CrossInstance**.

Requirement 3.8 (The oracle for the marking analysis). [3, Obl. 11.4] checks the marking against exhaustive scheduling at two and three instances. **f1r3comb-op**’s bounded exploration (Section 8) **MUST** serve as that oracle: given a unit, an instance count and a depth bound, it explores every interleaving, collects the encoded observations at quiescent markings, and reports whether they differ. It **MUST** classify $R = \text{for}(y \leftarrow a)y!(*y)$ as unsafe and $\text{for}(y \leftarrow a)(*y \mid *y)$ as safe [3, Req. 5.18], and **SHOULD** also be run on E7’s store, which neither **F1R3COMB** nor the draft has yet classified.

3.5 Provenance

Definition 3.9 (Provenance). As v0.2: every row carries **Prov**, **STATIC** on import; a produced row takes its premises’ common non-static tag, **STATIC** if all are static, **MIXED** otherwise; and a row naming a leaf takes that leaf’s root’s tag from the intern table’s **root** column. For a template instantiation the tag is the consumed address’s.

Requirement 3.10 (Provenance is metadata). As v0.2: it **MUST NOT** affect redexes, selection under **MaximalProgress**, or conclusions; it appears in traces as the root address’s content hash. **MIXED** rows are classified sanctioned or unsanctioned by Requirement 3.7.

4 The representation

v0.2’s representation is unchanged: structural interning with no hash on the hot path; **size**, **root** and **depth** columns; ids never leaving the process; the budget, including address depth; structure-of-arrays shape tables with a provenance column; import, export and round trip; the canonical **.cmat**. Three points are restated.

Requirement 4.1 (Structural interning). As v0.2, with the observation of E6 added: prefix sharing is by construction, and per-unfolding name cost is constant under both schemes (16 and 22).

Requirement 4.2 (Width). w is read from the header and instantiated for $w \in \{4, 5, 6\}$. Under the curried scheme and under `inst`, $w = 4$: base shapes have arity at most three, `consd` and `conss` four, the erecting rows three. $w = 5$ arises only under `literal-erection` with an (o5) occurrence, which is a negative control.

Requirement 4.3 (.cmat). As v0.2, with the header extended by the scheme, the marking, and the template names; `HOLE` relabelled to index 0.

5 The rule table

Requirement 5.1 (What the generator emits). As v0.2 — the seven routing and destructor rules, `quote`, `makel`, one rule per member of $\mathcal{F}(P)$ — and, per the header’s scheme, the erecting rule: under curried, `cons*` with one loud premise at slot 0 and conclusion `Encode{deliver: Slot(1), node: Instantiate{template: Slot(2), with: Payload(0)}}`; under `inst` the same with its own shape; under literal, nothing further, the family doing the erecting. Classes, join plans, the grading and w are derived from the emitted table.

Requirement 5.2 (Only a forwarder reads a store). As v0.2, a property of the generator.

Requirement 5.3 (Classes and plans are checked). As v0.2, per scheme: one class-(c) rule; one class-(b) rule per member of $\mathcal{F}(P)$, plus `makel` and the scheme’s erecting rule; seven class-(a) rules.

Requirement 5.4 (Grading). As v0.2. The erecting rules, like the family, have weight one.

6 Redex finding, selection, firing

The sparse layer, the three finders, the canonical key, the distinctness and symmetry masks, n -way joins over the generated signature, the conflict hypergraph, the `mix64` priority, the greedy maximal independent set and its equality with parallel rounds, the four resolvers including `CrossInstance`, race visibility, the order of effects, and debug-build soundness checking are unchanged from v0.2.

Requirement 6.1 (Order of effects). As v0.2: sort the step by canonical key; compute conclusions in that order, interning new names — including those an instantiation creates, in a fixed traversal of the template — in that order; remove consumed rows by stable compaction; append produced rows in the order computed. Parallel backends **MUST** assign ids as if sequentially.

Hazard 6.2 (Contention at statics, per scheme). Under `inst`, n instances contend at one static channel r per unit with a one-premise rule: n^2 redexes, a perfect matching in one step. Under curried the contention is spread over the duplicator tree and the `cons*` channels, each one-premise, so each is a perfect matching; the enumerated count is n^2 per erected atom. Under literal the multi-premise constructors enumerate n^m . The enumerated-to-fired ratio **MUST** be reported per static channel and per scheme.

7 Addresses and freshness

Requirement 7.1 (One allocator for both machines). As v0.2: `F1R3COMB`’s allocator [3, Req. 9.6], factored into `f1r3comb-term`.

Requirement 7.2 (Deploy checks). As v0.2, citing F1R3COMB v0.6: root address larger than every name of the image ([3, Req. 5.28]), static channels likewise ([3, Req. 5.31]), root addresses prefix-incomparable ([3, Req. 5.32]), with F1R3COMB’s diagnostics. Under `inst` the static channels are each unit’s r and f ; under `curried-erection` they also include every channel of the duplicator tree.

8 The operator layer

v0.2’s operator layer is unchanged in content — species, markings, successors with mass-action coefficients, exploration, the materialised T graded by W , closure, $T^\top \chi$ — and so is its scope: every compiled unit with an input contains `e`, so triangularity and closure apply to none of them, which F1R3COMB now reports too [3, Req. 10.2]. One use is added.

Requirement 8.1 (Bounded multi-instance exploration). `explore` MUST accept an instance count $k \in \{2, 3\}$ and a depth bound, deploy k instances at prefix-incomparable addresses, and enumerate every interleaving to the bound, returning the set of encoded observations at quiescent markings. This is Requirement 3.8’s oracle. The exploration is truncated by construction and MUST say so; it makes no triangularity or closure claim.

9 Instrumentation

Requirement 9.1 (Reports). As v0.2, with erection attributed per scheme — the `cons*` or `inst` firings and their `e`, and under `curried` the duplicator tree — and with names built per instance, sanctioned and unsanctioned mixed rows, and `curried` stores detected (Hazard 3.1).

Requirement 9.2 (The width measurement). `f1r3x comb-width` MUST report, per program, the width distribution under `inst` with erection separated from computation, and MAY report the `curried` scheme alongside for comparison. Under `inst` erection is two steps per instance; under `curried` it is the duplicator tree’s depth plus two. Whether either adds width without depth on real programs is the measurement [1, Obl. 13.7] asks for.

Requirement 9.3 (Subcommands). As v0.2, with `--scheme` on `comb-run -machine=matrix` and `comb-width`, and `--instances` on `comb-op`.

10 Architecture

The crates are v0.2’s: `f1r3comb-term` (F1R3COMB’s, plus the rule generator and the allocator), `f1r3comb-mat`, `f1r3comb-par`, `f1r3comb-op`, and the shared `f1r3comb-check`; none depends on `f1r3comb-ir`, `f1r3comb-lower` or `f1r3comb-run`. The test-only `literal-erection` feature MUST live in `f1r3comb-check`’s emitter and `f1r3comb-par`’s dev-dependencies only.

10.1 The device seam

Requirement 10.1 (Device kernels). As v0.2, with the erection fixed: a device backend MUST target $w = 4$ and the `inst` erection. Its substitution kernel is a gather over the template’s interned form with a batched insert-or-get of the results; its one erecting join is a one-premise match at each unit’s static r . Curried artefacts MUST NOT reach a device — they add a duplicator tree per unit, build more names (E6) and cannot erect every store (E7) — and neither may `literal-erection`. A curried artefact submitted to a device backend MUST be rejected with `comb-mat-scheme`. It is accepted only on identical step traces to `HostSequential` on the `inst` conformance corpus, and no work on it MAY begin before Obligation 11.7 is discharged. v0.2’s gate on phase two’s primitive is discharged by [4, §6.7]: the primitive is `inst`.

11 Correctness obligations and testing

Obligation 11.1 (Carried from v0.2). Round trip; finder agreement over every generated shape, now including `cons*`; Python parity on E1–E4; trace certification; operator correctness; determinism across backends, core counts and WebAssembly; the invariants of Requirement 5.4; device conformance.

Obligation 11.2 (The phase-two experiments, three arms). `f1r3comb-check` MUST reproduce `phase2-results.txt`: the E5 mixing rates for the literal scheme within a tolerance, and exactly zero for curried and `inst` at every instance count under `MaximalProgress`, sequential choice and `CrossInstance`; the E6 laws exactly (Requirement 3.6); and E7, zero for `inst` and a nonzero rate for the curried join. This is [3, Obl. 11.3] with the curried arm it lacked, and the literal scheme is its negative control.

Obligation 11.3 (The W regression and the deciding terms). W [3, Obl. 11.1], its three-instance variant, and both halves of [3, Req. 5.18], run under `MaximalProgress` and `CrossInstance` for 100 seeds each, under each conforming scheme: the observation at $\llbracket @X \rrbracket$ MUST match B3LL0WS’s on every seed and no row MUST be unsanctioned-mixed. Under `literal-erection` the rate of the crossed residue is recorded as a negative control.

Obligation 11.4 (Both schemes, and contexts round-trip). [3, Obl. 11.5, Obl. 11.6] on this machine: compile the corpus under `inst` and under `curried-erection`, run both, and compare observations; and for every unit, compare the atoms `inst` erects with the atoms the curried erection produces when run to completion. Units in the class of Hazard 3.1 MUST be reported separately, since the curried build cannot erect them within the discipline and a divergence there is the scheme’s, not the implementation’s.

Obligation 11.5 (The marking oracle). Requirement 3.8 run on F1R3COMB’s marking corpus; any disagreement with F1R3COMB’s analysis MUST be reported to F1R3COMB as a counterexample [3, Obl. 11.4].

Obligation 11.6 (Addresses and differential testing). As v0.2, citing [3, Obl. 11.9, 11.11, 11.14]. F1R3COMB now emits both conforming schemes, so v0.2’s test-only `inst` emitter is retired.

Obligation 11.7 (The width of real programs). [1, Obl. 13.7], on `inst` artefacts, with erection separated. Gates the device seam.

12 Errors and diagnostics

As v0.2, with three additions:

Code	Where	Meaning
<code>comb-mat-template</code>	import, step	A <code>cons*</code> template outside the curried validity class (Requirement 3.2)
<code>comb-mat-curried-store</code>	deploy	Warning: a curried unit whose store payload needs a join (Hazard 3.1)
<code>comb-mat-scheme</code>	deploy	A scheme the backend does not carry: curried or literal on a device (Requirement 10.1)

13 Performance, portability, determinism

As v0.2, with the targets stated for `inst`: 10^4 unfoldings of D_x with no rise in per-unfolding step time; 64 concurrent instances of a 10^3 -atom unit erected in two steps. For comparison only, the curried scheme would take $\lceil \log_2(\text{atoms}) \rceil + 2$.

14 What the implementation asks of the papers

Discharged since v0.4. F1R3COMB v0.6 makes `inst` its default [3, Dec. 5.19], which was v0.4’s first request; its mixing obligation now asserts `inst`’s figures [3, Obl. 11.3], which was the second; `inst`, the context node and the hole have tags [3, Req. 4.11]; and the question of which scheme the logic reads is now stated by F1R3COMB in its sharpest form [3, §14, item 5]. Earlier discharges are as recorded in v0.3 and v0.4.

Open.

1. **Commit the three-arm measurements.** [3, Rem. 5.20] and [3, §14, item 2] still say the curried arm does not exist. It does: `rhocombmat.py` v1.3, `phase2.py` and `phase2-results.txt` accompanying this specification run E5–E7. Proposed: commit them, and replace Remark 5.20 with the figures of Section 3.
2. **F1R3COMB v0.6, Remark 5.22, depth.** The argument that decides F1R3COMB’s choice is stated as keeping “the band of the upper-triangular matrix shallow and constant”, with the curried scheme costing n steps for an n -atom unit. Two corrections, neither of which weakens the choice. Every compiled program with an input contains `e`, so it has no upper-triangular transfer operator ([1, Thm. 6.1(ii), §14.2]), which F1R3COMB itself records [3, Req. 10.2]; the depth that matters is the number of bulk-synchronous steps. And in that unit the curried scheme costs $\lceil \log_2 n \rceil + 2$ steps, not n , because its n constructors fire in parallel after the duplicator tree (E5: 3 against 2; E6: 10 against 7). Proposed: restate the argument in steps, with the measured figures.
3. **F1R3COMB v0.6, Remark 5.22, “a fixed linear map”.** Right in the sense that matters: every rule is a ladder-operator monomial schematic in names, and $C[v]$ is a class-(b) encode like any constructor’s, a function on name indices ([1, §5.1]). Proposed: say so in those terms, since “fixed linear map on the vector encoding” suggests the state vector, which $C[v]$ does not act on directly.
4. **F1R3COMB v0.6, Remark 5.22, “one rule row”, and open question 5.** F1R3COMB argues that `inst` is one rule row where the curried family needs a row per shape and path tuple, and then notes that one row per context makes the rule matrix program-dependent after all. In this representation neither holds: the template is the third column of the row, a name in the state, so `inst` is one rule for every program and `cons*` is one rule too (Decision 2.6). The rule table is program-independent under both schemes. Proposed: close open question 5’s second half on that basis, and rest the fourth argument on the step count rather than the rule count.
5. **F1R3COMB v0.6, Remark 5.22, “what does not change”.** On this machine the work figure does change: `inst` builds 16 names per unfolding of D_x against curried’s 22, because structural interning hashes nothing and the substitution visits only hole-bearing names (Requirement 3.5). The requirement to charge by size [3, Req. 5.25] stands and is met by reporting names built.
6. **Hole indices** [3, Req. 5.24]. The hole tag carries a canonical index, but `inst` has one input, so every hole is filled with the same name and the index distinguishes nothing. It does cost sharing: two occurrences of one leaf beneath different indices encode, hash and intern as different names. F1R3COMB-MAT collapses the indices on import (Requirement 3.3). Proposed: an unindexed hole, unless a multi-input context is intended, in which case say so.
7. **The curried scheme’s reach**, now load-bearing (Hazard 3.1). Under v0.6 the curried build is the logic tooling’s target [3, Req. 5.27] and [3, Obl. 11.5, Obl. 11.6] run it on the whole

corpus. Units whose gate stores mention scoped names in two atoms cannot be erected by it within the discipline (E7), so those obligations will fail on them for reasons of the scheme, and [3, Obl. 11.6]’s round trip is defined only where the curried erection reaches. Proposed: state the limit in draft 3’s §6.7 and F1R3COMB’s Requirement 5.27, exclude or flag such units in both obligations, and count it in the decision of [3, §14, item 5]: a logic read over the curried build does not see those programs at all. **cons*** also has no tag in [3, Req. 4.11].

8. **F1R3COMB v0.6, Requirement 9.6** still describes the root address as “of a shape outside that program’s family”; freshness is by size (Req. 5.28, [4, Lem. 6.14]). Carried.
9. **F1R3COMB v0.6, Requirement 10.4** still cites [5]’s weighted count as “Def. 6.6”; it is Def. 4.2 and Prop. 6.17. Carried.
10. **F1R3COMB v0.6, Requirement 9.4** (decode by projection): scope it to builds where **presentation-b** is reachable. Carried.
11. **The allocator in f1r3comb-term, and a Merkle form of the content hash** for $O(1)$ hashing of new addresses on **f1r3comb-run**. Carried.

15 Open questions

1. **The logic and inst** [3, §14, item 5]: a structural production for the erecting row, or a logic of the curried build that does not see E7’s class of programs.
2. **Width under inst** on real programs, with erection separated: does two-step erection add width without depth, as E5 and E6 predict?
3. **Cost accounting for inst**: charging by names built rather than by step, as [3, Rem. 5.22] anticipates.
4. **Resolver fidelity and fairness** ([1, Obl. 13.1, Obl. 13.6]).
5. **Address compaction** [3, §15, item 1].
6. **Garbage, the history unfolding, weights**: as v0.2.

16 Delivery

M	Deliverable	Done when
X0	<code>f1r3comb-mat</code>	As v0.2, with templates and <code>HOLE</code>
X1	Finders	As v0.2, over the generated signature including the erecting rows
X2	<code>f1r3comb-par</code> , sequential	Resolvers; the instantiation kernel, <code>inst</code> first, then the curried validity check; deploy checks; marking read from the header; the E6 laws, <code>inst</code> primary
X3	Phase-two evidence	The three arms of Obligation 11.2; <code>W</code> and the deciding terms under <code>inst</code> , and under curried for comparison; reported to F1R3COMB with the requests of Section 14, items 1 and 7
X4	The marking oracle	Multi-instance bounded exploration (Requirement 8.1); Obligation 11.5 on F1R3COMB's corpus
X5	The width gate	Under <code>inst</code> , erection separated; needs F1R3COMB M2. Gate on X8
X6	Threaded host	Determinism across cores and <code>wasm32</code>
X7	<code>f1r3comb-op</code>	As v0.2, scoped
X8	Device conformance	<code>inst</code> only; external backend only after Obligation 11.7

17 Changes from v0.4

v0.4	v0.5
Companion to F1R3COMB v0.5, which defaulted to curried	Companion to F1R3COMB v0.6, which defaults to <code>inst</code> ; citations renumbered
<code>inst</code> behind <code>inst-erection</code> , enabled by default	<code>inst</code> unconditional; curried behind <code>curried-erection</code> , F1R3COMB's name
Curried support for compatibility, to be retired	Curried support for F1R3COMB's cross-scheme obligations and the logic tooling's build
<code>HOLE</code> with a tag to be allocated	F1R3COMB's tags for <code>inst</code> , context and hole; hole indices collapsed on import, renumbered on export (Requirement 3.3)
Substitution over every name of the template	Over hole-bearing names only, precomputed per template
—	Contexts are data: <code>inst</code> and <code>cons*</code> each one rule for every program (Decision 2.6)
—	Obligation 11.4: both schemes, and contexts round-trip, on this machine
Eleven asks, the first to make <code>inst</code> F1R3COMB's default	Discharged; eleven open, six new, on v0.6's rationale, hole indices, and the curried reach

v0.3's and v0.4's changes carry forward.

References

- [1] L. G. Meredith. *Rho combinators as sparse linear algebra*. Version 1.2, `publications/rho-combinators/rhocomb-matrix`, with `rhocombmat.py`, `experiments.py` and `phase2.py`, September 2026.
- [2] F1R3FLY.io. *F1R3Comb-Mat: the rho combinators as sparse linear algebra*. Specification v0.2, `publications/rho-combinators/f1r3comb-mat-spec-v02`, 23 September 2026.
- [3] F1R3FLY.io. *F1R3Comb: a compiler from kernel rho to the name-free rho combinators, and a machine that runs the result*. Specification v0.6, `publications/rho-combinators/f1r3comb-spec-v06`, 23 September 2026.
- [4] L. G. Meredith and M. Stay. *Name-free combinators for the rho calculus: a repair, a linear encoding, and the one operation that was missing*. Draft 3, `publications/rho-combinators`, September 2026.
- [5] F1R3FLY.io. *A spatial-behavioural logic for the rho combinators*. Draft 2, revision 2, `publications/rho-combinators/rhocomb-logic`, 23 September 2026.
- [6] F1R3FLY.io. *K1ndllng: a parser and normaliser for kernel rholang*. `publications/campf1r3/k1ndllng-spec`, 2026.